# TLS 1.3 CRYPTOGRAPHY & SECURITY CASE STUDY REPORT

**COURSE** : CSA0712 COMPUTER NETWORKS

**NAME**:KRISHNAPRIYA P

**REG NO**: 192511279

CO4-AT2- Industry Problem Solving Task

## 4.1 The 1-RTT Handshake

### What is a Round-Trip Time?

A **Round-Trip Time (RTT)** means:The time taken for a message to travel from the client to the server and for the response to come back to the client.In TLS 1.3, a secure connection can normally be established in **one round trip**.

### How TLS 1.3 achieves 1-RTT

The main improvement is that the client sends much more cryptographic information in its first ClientHello message.

The ClientHello contains information such as:

- Supported TLS versions
- Supported cipher suites
- Supported key exchange groups
- Random value
- Key Share

### TLS 1.3 1-RTT Handshake Diagram



**What crucial guess does the client make?**

- The client makes an important assumption or guess:
- It predicts which key exchange group or cryptographic algorithm the server supports.

**4.2 Zero Round-Trip Time (0-RTT) Mode**

- **0-RTT (Zero Round-Trip Time)** allows a **returning client** to send encrypted application data immediately, without waiting for a response from the server.
- For example, a user who previously visited a website can reconnect and immediately send : GET /news

**How does Session Resumption work?**

Imagine that a user visits a website for the first time.

**First Connection**

CLIENT SERVER

| |

|--------- Full TLS Handshake ---------->|

| |

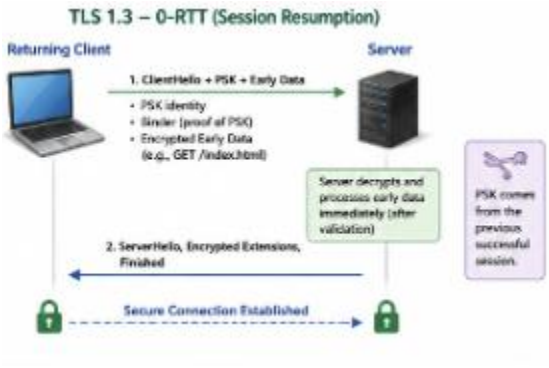
|<-------- Secure Connection ----------|

| |

|====== Encrypted Communication ==========>|

During or after this secure connection, the client receives information that can be used for session resumption.Later, when the user visits the same server again, the client can resume the session.

**0-RTT Connection**




**What is a Pre-Shared Key (PSK)?**

A **Pre-Shared Key (PSK)** is secret cryptographic information already known to both the client and server.

In TLS 1.3 session resumption:

1. The client previously established a secure TLS connection.
2. The server provides resumption information.
3. Both sides can later use related secret keying material.
4. The returning client uses the PSK during the next connection.
5. The PSK helps derive encryption keys for **0**-RTT Early Data.

**4.3 Security Trade-off: Perfect Forward Secrecy**

Although 0-RTT is very fast, it has an important security trade-off.The first 0-RTT message does not receive the same fresh Perfect Forward Secrecy protection as normal application data sent after a new key exchange is completed.

**What is Perfect Forward Secrecy?**

**Perfect Forward Secrecy (PFS)** means that even if an attacker obtains certain long-term secrets in the future, previously recorded communications should remain protected because each connection uses fresh, temporary cryptographic secrets.

**Normal TLS 1.3 Connection**

Connection 1

Fresh Secret A

↓

Encrypted Data A

Connection 2

Fresh Secret B

↓

Encrypted Data B

Connection 3

Fresh Secret C

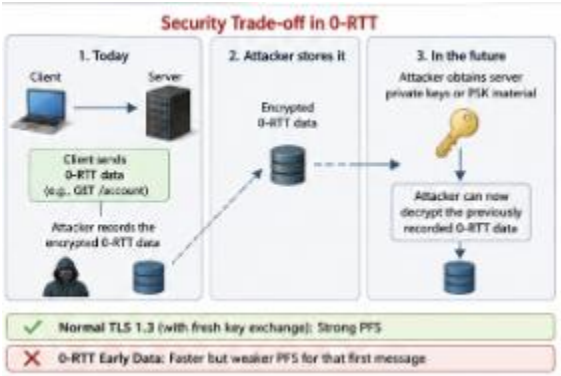
↓

Encrypted Data C

Each connection uses fresh key exchange material.Therefore, compromising one secret should not automatically expose all previous sessions.

**Security Problem with 0-RTT**

In 0-RTT mode, the client sends encrypted data before completing a fresh key exchange for the new connection.The early data is protected using keys derived from the previous session's resumption information or PSK.




## 5: The Replay Attack Threat

### 5.1 Anatomy of a Replay Attack

- A Replay Attack occurs when an attacker captures a valid message and sends the same message again to make the server repeat the original action.
- In TLS 1.3 0-RTT mode, a returning client can send encrypted early data immediately using information from a previous session. Because the data is sent before a fresh handshake is fully completed, an attacker who captures a valid 0-RTT transmission may be able to copy and replay it. The attacker does not need to decrypt or understand the message; they only need to resend the captured valid request.

For example, on a **public university Wi-Fi network**:

1. A student connects to an online website using TLS 1.3 session resumption.
2. The browser immediately sends a 0-RTT request.
3. An attacker monitoring the same network captures the encrypted packet.
4. The original request reaches the server and is processed.
5. The attacker resends the exact captured packet.
6. If the server does not detect the duplicate, it may process the same request again.

**5.2 Safe vs. Unsafe Actions**

- Not every replayed request causes the same level of damage. The risk depends on whether the request **only reads information** or **changes the state of the server**.
- **Safe example: GET /news**
- A request such as: **GET /news** only asks the server to send information. If an attacker replays the request several times, the server may simply send the same news data again.

Therefore:

- No purchase is made.
- No money is transferred.
- No important data is changed.
- No duplicate transaction occurs.

Such operations are generally called **idempotent or safe-to-repeat operations** when repeating them does not create an additional harmful effect.

**Unsafe example: POST /api/buy-item**

Now consider:**POST /api/buy-item**

This request changes the state of the server by creating an order or purchasing an item. If an attacker captures and replays the request:

Original request → Item purchased once

Replay 1 → Item purchased again

Replay 2 → Another order created

Replay 3 → Another charge or transaction

This can cause:

- Duplicate orders.

- Multiple payments.
- Financial loss.
- Duplicate database records.
- Unauthorized repeated actions.

Therefore, state-changing and non-idempotent operations are dangerous in unrestricted 0-RTT mode.

**How web developers should protect their APIs**

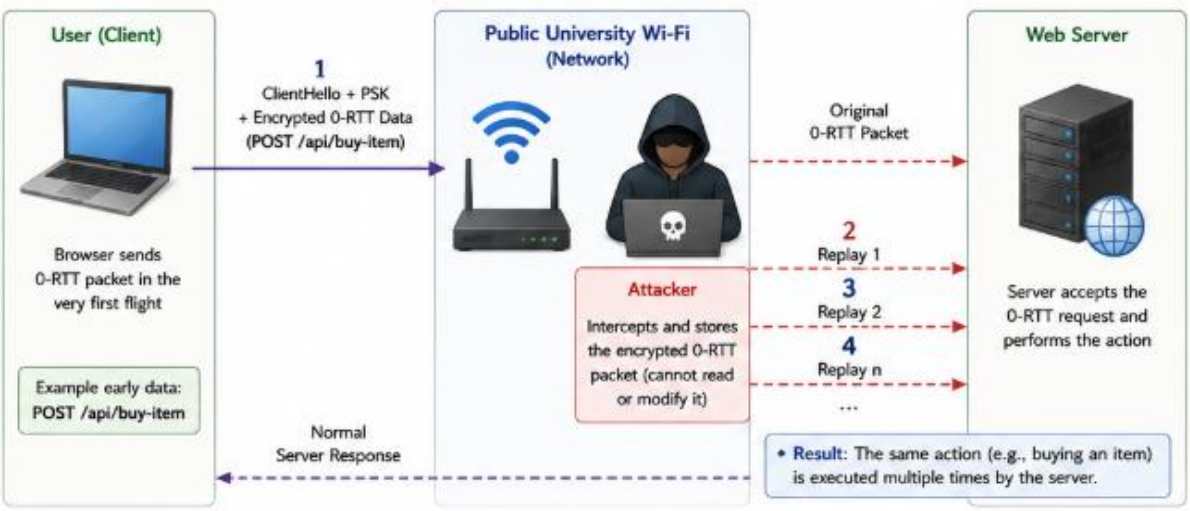
Web developers should use several protections:

1. **Do not allow sensitive operations as unrestricted 0-RTT requests.**
   Payments, purchases, money transfers, password changes, and account modifications should normally wait until the regular TLS handshake is established.
2. **Use idempotency keys.**
   Each important request can contain a unique identifier. If the same identifier appears again, the server recognizes it as a duplicate and does not perform the action twice.

Example:

Idempotency-Key: ORDER-12345

3. **Use anti-replay mechanisms.**
   The server can track previously processed requests, tickets, or request identifiers and reject duplicates.
4. **Use timestamps and expiration periods.**
   A request should only be accepted within a limited time window, reducing the opportunity for an attacker to replay it later.
5. **Design APIs to be idempotent where possible.**
   Repeating the same request should produce the same result instead of repeating the transaction.




## 6: Cryptographic Cleaning & Protocol Downgrades

### 6.1 Deprecation of Weak Ciphers

**What is cryptographic cleaning?**

Cryptographic cleaning means removing old, weak, or unnecessary cryptographic algorithms from a security protocol instead of continuing to support them for compatibility.TLS 1.2 supported a large number of cryptographic options. Some of these became outdated as attacks and weaknesses were discovered. Examples include:

- Static RSA key exchange
- Older hashing algorithms such as MD5
- Other legacy cryptographic constructions
- Weak or obsolete cipher combinations

TLS 1.3 significantly reduces these older choices and focuses on modern cryptographic techniques.

**1. Problem with Static RSA Key Exchange**

- Static RSA uses the server's long-term private key to protect session secrets. If this key is stolen later, previously recorded communication may be at risk.

- Example: An attacker records a user's online banking session today. If they later obtain the server's RSA private key, they may be able to decrypt the recorded session.
- TLS 1.3 removes Static RSA and uses ephemeral key exchange, providing better Perfect Forward Secrecy (PFS).

**Security benefit**

If the server's long-term private key is compromised in the future, it should not automatically allow an attacker to decrypt previously recorded sessions that used fresh ephemeral key exchange.

This provides **Perfect Forward Secrecy (PFS)**.

**2. Problems with Weak Hashing Algorithms**

Older cryptographic designs included algorithms such as **MD5**, which is considered cryptographically broken for collision resistance.A collision occurs when an attacker finds two different inputs that produce the same hash value.

Message A ——→ Hash ——→ ABC123

Message B ——→ Hash ——→ ABC123

- If an attacker can deliberately create such collisions, the integrity and authenticity mechanisms based on that hash may become unreliable.
- The security problem is not simply that an old algorithm is slower or outdated. The real danger is that attackers may use known mathematical weaknesses to attack systems that still rely on it.

**Why completely remove weak algorithms?**

A major security benefit of TLS 1.3 is that weak algorithms are not simply marked as “optional” or “not recommended.”They are removed from normal TLS 1.3 negotiation

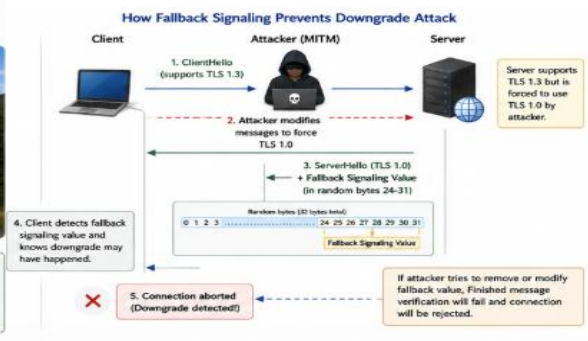
If weak algorithms remain available, problems can occur because:

- An administrator may configure the server incorrectly.
- An old client may request a weaker option.
- Compatibility settings may accidentally enable weak algorithms.
- An attacker may attempt to force a weaker algorithm.
- Developers may choose insecure settings for convenience.

**Main security benefits of removing weak algorithms**

| **Security Benefit** | **Explanation** |
|---|---|
| Reduced attack surface | Attackers have fewer weak algorithms to target. |
| Stronger default security | Connections automatically use modern cryptography. |
| Fewer configuration mistakes | Administrators cannot accidentally select removed TLS 1.3 algorithms. |
| Better forward secrecy | Ephemeral key exchange protects previous sessions from later key compromise. |
| Reduced downgrade opportunities | Fewer weak alternatives are available for negotiation. |
| Simpler implementation | Developers and administrators have fewer insecure compatibility options to manage. |



### 6.2 Preventing Downgrade Attacks

**What is a Downgrade Attack?**

A **Downgrade Attack** occurs when an attacker tries to force two modern systems to communicate using an **older and weaker protocol version**.

For example:

- Client supports TLS 1.3
- Server supports TLS 1.3
- Attacker interferes with the handshake
- Client and server are tricked into using TLS 1.0

The attacker does this because older protocols may contain weaker cryptographic mechanisms or known security vulnerabilities.

**Example of a Man-in-the-Middle Attack :** A client requests TLS 1.3, but the attacker alters the connection to make both sides use the older TLS 1.0 protocol. This is called a downgrade attack**.**

**How TLS 1.3 protects against downgrade attacks**

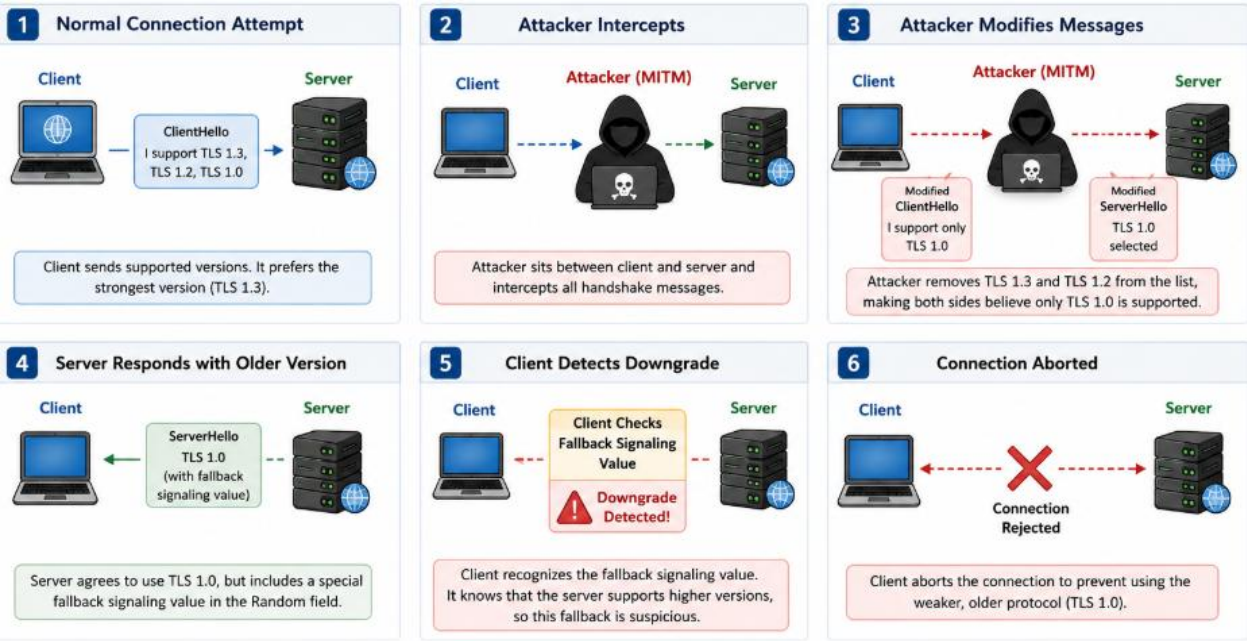
TLS uses **downgrade signaling** in defined fallback situations.When a TLS 1.3-capable server negotiates an older TLS version under circumstances that could indicate a downgrade, the server includes a special recognizable value in the **ServerHello random field**.This acts as a warning signal.The client can detect:“The server supports a newer version, so this fallback to an older protocol may be suspicious.”.A TLS 1.3-capable client that sees the appropriate downgrade signal can reject the connection instead of silently continuing with the weaker protocol.

**Handshake integrity also provides protection**

- TLS does not rely only on the downgrade signal.
- During the handshake, important handshake messages are included in a **cryptographically protected transcript**. The Finished messages verify the integrity of the handshake.

- This means that an attacker cannot freely change important handshake information without causing verification to fail.

**Complete Downgrade Attack Scenario**




**Security Benefits**

TLS 1.3 downgrade protection provides the following advantages:

**1. Prevents forced use of weak protocols**

An attacker cannot easily force modern systems to silently use TLS 1.0 or another older protocol.

**2. Detects suspicious fallback**

Special downgrade signaling allows capable clients to recognize when a server that supports newer TLS appears to have been forced into an older version.

**3. Protects handshake integrity**

Cryptographic verification of the handshake transcript makes undetected modification of protected negotiation data much more difficult.